Sprint 3 - S3_02
Paradigmas: Recursão e Funções de Alta Ordem

Matheus Moreli Durão Silva
Matrícula 12621CBS217
Cibersegurança - Bacharelado - Noturno

2026

Entregável

Algoritmos recursivos e comparação com abordagens iterativas e funcionais. Este documento registra o objetivo do card, a solução desenvolvida e a forma de conferência. Os arquivos de código e as evidências citadas permanecem na mesma pasta deste documento.

O que foi estudado e aplicado

- Caso-base e chamada recursiva
- Fibonacci com memoização
- Uso de lambda, map e filter
- Comparação de resultados e limites da recursão

Como conferir

O avaliador pode abrir os arquivos desta pasta, executar os exemplos quando houver código e comparar a saída com a explicação apresentada. Nos cards ligados ao Inventário de Ativos, a solução foi relacionada ao Trabalho 2 sem modificar requisitos já atendidos do CRUD.

Comparação medida

O script `comparar_desempenho.py` registrou a mediana de cinco execuções em Python 3.13.15/Windows. Para fatorial de 800, iteração levou 0,074453 ms/chamada e recursão 0,114057 ms/chamada; ambas têm tempo $O(n)$, mas a recursão acrescenta pilha $O(n)$. Para Fibonacci de 30, a versão sem cache levou 66,0056 ms/chamada e a memoizada 0,0037 ms/chamada. Sem cache há recomputação de subproblemas; com cache, tempo e memória são $O(n)$.

Funções de alta ordem

`map` extrai nomes, `filter` seleciona ativos de criticidade Alta/Crítica, `reduce` soma comprimentos e `sorted` com `lambda` ordena por nome. A tabela completa, o método de medição e suas limitações constam em `analise_comparativa.md`; os valores absolutos podem mudar em outra máquina.

Conclusão

Os exemplos mostram que recursão pode ser clara, mas memoização é decisiva em problemas com subproblemas repetidos.